GUIA VISUAL 001

# O QUE É LINUX

*Servidor desconhecido, 3h20 da manhã:* `uname -a` *e* `/etc/os-release` *respondem perguntas diferentes.*

**001**
GUIAS LINUX

## LINUX É O KERNEL — O RESTO É INTEGRAÇÃO

ESPAÇO DE USUÁRIO

**APLICAÇÕES**
navegador · nmap · Python · o seu script

**UTILITÁRIOS E SHELL**
bash · coreutils · systemd · apt

**BIBLIOTECAS**
glibc · OpenSSL · libcurl

chamadas de sistema

ESPAÇO DE KERNEL

**KERNEL LINUX**
processos · memória · arquivos · rede · drivers

**HARDWARE**
CPU · RAM · disco · placa de rede

### O KERNEL DEFINE

Drivers e suporte a hardware, escalonamento de CPU, memória, sockets. **Nenhum programa toca o hardware** — todos pedem ao kernel. Monolítico e **modular**: `lsmod` mostra o que foi carregado sem reiniciar.

### A DISTRIBUIÇÃO DEFINE

Biblioteca C, init, shell, **gerenciador de pacotes**, repositório, instalador e a política de atualização. É onde nasce o `apt` × `dnf` — e a janela de suporte.

### OS NÚMEROS NÃO CONVERSAM

Kernel `6.12.25` e distro `2025.2` não têm relação. Ler os dois é o **primeiro diagnóstico** de qualquer máquina desconhecida.

## AS DUAS PERGUNTAS DO PRIMEIRO MINUTO

## O TRONCO DEFINE OS COMANDOS

| FAMÍLIA | PACOTE · GERENTE | DERIVADAS | ONDE APARECE |
|---|---|---|---|
| **Debian** (1993) | `.deb` · `apt` | Ubuntu, **Kali**, Mint, Raspberry Pi OS | Servidor pequeno/médio — a família deste guia |
| **Red Hat** (1994) | `.rpm` · `dnf` | Fedora, RHEL, Rocky, Alma | Datacenter corporativo (firewalld, SELinux) |
| **Arch** (2002) | `.pkg` · `pacman` | Manjaro | *Rolling* minimalista, montado à mão |
| **Alpine** | `.apk` · `apk` | — | Contêiner: usa `musl` no lugar da `glibc` |

## O QUE SUSTENTA TUDO: AS 4 LIBERDADES

**0** Executar para **qualquer** fim

**1** Estudar e adaptar (exige o fonte)

**2** Redistribuir cópias

**3** Distribuir versões modificadas

### RESUMO RÁPIDO

- ✓ Linux, no sentido estrito, é **só o kernel**; a distro é o kernel +

### NO KALI

**Debian rolling** da OffSec, saído do ramo `testing`. O `2025.2`

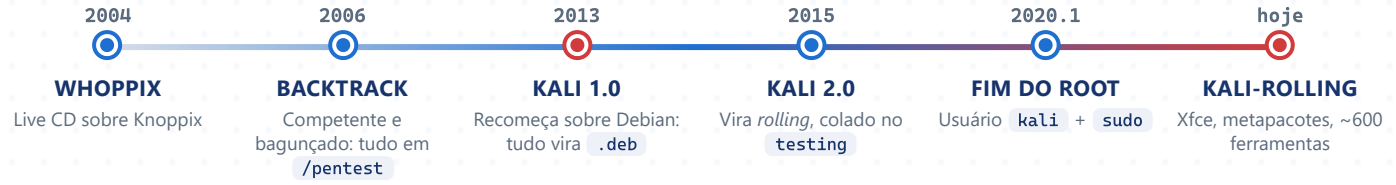
GUIA VISUAL 002

# POR QUE KALI LINUX

*Instalou Kali no notebook de trabalho e o Wi-Fi morreu no `full-upgrade`? Não é defeito — é uso fora de propósito.*

002
GUIAS LINUX

## VINTE ANOS ATÉ O `KALI-ROLLING`




## AS DECISÕES DE PROJETO QUE EXPLICAM O COMPORTAMENTO

### REPOSITÓRIO ÚNICO E ASSINADO

Poucos espelhos, chave GPG do projeto. **Nunca adicione repositório de outra distro** — quebra dependência e destrói a procedência.

### CURADAS, NÃO EXAUSTIVAS

~600 ferramentas que passam por triagem (é mantida? licença? duplica?). O oposto do BlackArch, que empacota milhares.

### KERNEL COM PATCHES

O sufixo `-kali1` traz **injeção de pacotes e modo monitor** em Wi-Fi. É o que você não obtém instalando as ferramentas à mão.

### METAPACOTES

`kali-linux-core` → `default` → `large` → `everything`. Só listas de dependências: VM de 2 GB ou estação de 30 GB.

**DISTRIBUIÇÃO DE SEGURANÇA ≠ DISTRIBUIÇÃO SEGURA**

Uma instalação padrão traz centenas de binários ofensivos e defaults voltados à conveniência do operador — a superfície de ataque é grande **por construção**. Se comprometem sua máquina Kali, o atacante acha a caixa de ferramentas montada e aquecida. Por isso: VM com snapshot, nada exposto à internet, e um cliente nunca encosta no outro.

**O que legitima o uso é a autorização, não a técnica.** Sem escopo por escrito e autorização de quem pode dá-la, a mesma ação é crime no Brasil (Lei 12.737/2012).

## A MÁQUINA É PARA QUÊ?

### USE KALI

- ✓ Teste de invasão **autorizado** e resposta a incidente.
- ✓ Forense digital e engenharia reversa.
- ✓ Auditoria de rede sem fio (é onde o kernel dele brilha).
- ✓ Laboratório: CTF, máquinas vulneráveis, aula prática.
- ✓ Sempre que possível: **em VM com snapshot**.

### USE OUTRA

- ✓ Servidor → **Debian estável, Ubuntu LTS, Rocky**. Rolling não tem janela de suporte.
- ✓ Desktop de trabalho → Ubuntu, Fedora, Mint.
- ✓ Só aprender Linux → Debian ou Ubuntu em VM.
- ✓ Se você não sabe o que a ferramenta faz — ou se a empresa proíbe.
- ✓ Vizinhas úteis: Parrot, BlackArch, Tails, REMnux.

### RESUMO RÁPIDO

- ✓ Debian *rolling* da OffSec para pentest, forense e auditoria — herdeiro do BackTrack.
- ✓ 2013 reconstruiu sobre Debian para ganhar **empacotamento**

### NO KALI

Atualize sempre com `apt update && apt full-upgrade` — o `apt upgrade` se recusa a remover pacotes e deixa a máquina meio atualizada, o pior estado possível num rolling. Se o `apt`

GUIA VISUAL 003

# INSTALANDO O KALI

*VM, bare metal, live USB, WSL ou contêiner — os cinco caminhos servem a problemas diferentes.*

003
GUIAS LINUX

## ANTES DE TUDO: A IMAGEM É MESMO DO KALI?

**1 — IMPORTE A CHAVE**

`wget -qO- archive.kali.org/archive-keyring.gpg | gpg --import`

→

**2 — BAIXE AS SOMAS**

`SHA256SUMS` e `SHA256SUMS.gpg`, ao lado da imagem.

→

**3 — CONFIRA A ASSINATURA**

`gpg --verify SHA256SUMS.gpg SHA256SUMS` → tem de dizer **Good signature**.

→

**4 — CONFIRA A SOMA**

`sha256sum -c SHA256SUMS --ignore-missing` → **OK**. Só então grave.

**Conferir só a soma não prova nada** — se o espelho foi adulterado, a soma veio adulterada junto. A assinatura GPG é o elo que fecha a cadeia. O aviso `key is not certified` é normal; `BAD signature` não é: descarte e baixe de novo.

## OS CINCO CAMINHOS

| MÉTODO | ENTREGA | COBRA | USE QUANDO |
|---|---|---|---|
| **Máquina virtual** ← o padrão | Isolamento, **snapshot**, uma VM por cliente | Wi-Fi só por USB externo repassado; perde desempenho | Estudo e quase todo trabalho profissional. 4 GB RAM · 2 vCPU · **60–80 GB** |
| **Bare metal** | Desempenho total, GPU para `hashcat`, Wi-Fi interno | Sem snapshot; particionar apaga o resto | Máquina dedicada à função — nunca o notebook de uso geral |
| **Live USB** + persistência | Não escreve no disco da máquina; persistência com LUKS | Lento; gravar a mídia é a operação mais perigosa | Forense, recuperação, trabalho em máquina de terceiros |
| **WSL2** (Windows) | Rápido, integra com o Windows; `kali-win-kex` dá tela | **Kernel é o do WSL**: sem patches, USB irregular, sem modo monitor | Linha de comando, scripts, análise de arquivo. Bom complemento, mau substituto |
| **Contêiner** | Sobe em segundos, descartável, ideal em pipeline | Usa o kernel do hospedeiro; péssimo como ambiente interativo | Rodar *uma* ferramenta pontual sem instalar nada |

## NA VM, O MODO DE REDE É DECISÃO DE SEGURANÇA

### NAT · O PADRÃO

VM → host → LAN

Sai para a internet escondida atrás do hospedeiro; **ninguém de fora inicia conexão**. Serve para atualizar e navegar.

### BRIDGE · CUIDADO

VM → LAN real

IP próprio no roteador, visível como outra máquina. Necessário quando a VM precisa ser alcançada — e é exatamente o que a política de TI proíbe numa rede corporativa.

### HOST-ONLY · O LABORATÓRIO

VM ⇔ alvo ⇔ sem saída

Rede fechada entre as VMs. **Máquina-alvo vulnerável nunca em NAT nem em bridge** — senão seu laboratório vira porta de entrada.

GUIA VISUAL 004

# PRIMEIRO CONTATO

*Não há "Meu Computador" nem letra de unidade. Não é falta — é outro modelo, e ele começa no firmware.*

004
GUIAS LINUX

## DO BOOT AO PROMPT: QUEM ENTREGA A TELA

**FIRMWARE** UEFI / BIOS → **GRUB** escolhe o kernel → **KERNEL** monta a raiz → **SYSTEMD** PID 1, sobe tudo → **GRAPHICAL .TARGET** o alvo do systemd → **LIGHTDM** tela de login → **PAM** autentica → **SESSÃO XFCE** painel, janelas → **TERMINAL** o seu prompt

**A camada gráfica é a única dispensável.** `systemctl get-default` mostra o alvo; `set-default multi-user.target` entrega um sistema completo sem tela — é o que se faz em servidor. E a saída de emergência quando o Xfce trava é **Ctrl+Alt+F3**: um console virtual ligado direto ao kernel. **Ctrl+Alt+F7** (ou F2) volta — a sessão gráfica continuou rodando o tempo todo.

## LENDO O PROMPT CARACTERE A CARACTERE

```
┌──(kali㉿kali)-[~]
└─$
```

Vermelho depois de um comando? Ele falhou — o Zsh do Kali colore o prompt pelo código de saída, poupando um **echo $?** a cada passo.

- **kali** — usuário logado
- **kali** — nome da máquina
- **~** — diretório atual = `/home/kali`
- **$** — comum · **#** = **root**, cada comando pode destruir o sistema

## XFCE NÃO É UM PROGRAMA — SÃO PEÇAS QUE SE REINICIAM SOZINHAS

| PROCESSO | O QUE DESENHA | SINTOMA DE QUE CAIU |
|---|---|---|
| `xfwm4` | Bordas, botões, qual janela fica na frente | Janelas sem barra de título → `xfwm4 --replace &` |
| `xfce4-panel` | A barra: menu, lista de janelas, relógio, bandeja | Sumiu o painel e o ícone de rede |
| `xfdesktop` | Papel de parede e ícones da área de trabalho | Fundo preto, sem ícones |
| `xfsettingsd` | Tema, fontes, cursor, teclado e mouse | Volta ao tema cinza padrão |
| `xfce4-session` | Coordena tudo e restaura o estado ao relogar | Nada restaura entre logins |

## OS QUATRO AJUSTES DO PRIMEIRO DIA

### RESUMO RÁPIDO

### NO KALI

GUIA VISUAL 005

# ESTRUTURA DE DIRETÓRIOS (FHS)

*Raiz com 96% de uso às 23h: se é `/var/log` a história é uma, se é `/var/cache/apt` é outra.*

005
GUIAS LINUX

## UMA ÁRVORE SÓ, SEM LETRA DE UNIDADE

| | |
|---|---|
| `/boot` | Kernel, initrd e o GRUB. Enche depois de algumas atualizações → `apt autoremove` |
| `/usr` | Programas, bibliotecas e dados estáticos. `/bin`, `/sbin` e `/lib` são só links para cá |
| `/usr/local` | O que **você** compilou. O `apt` nunca escreve aqui |
| `/etc` | Configuração, **sempre em texto**, nunca binário. Padrão Debian: fragmentos em `*.d/` |
| `/home` `/root` | Diretórios pessoais. O root fica fora de `/home` — que pode falhar ao montar |
| `/opt` | Software de terceiros autocontido, um diretório por produto |
| `/srv` | Dados servidos a terceiros (no Debian, na prática, `/var/www`) |
| `/mnt` `/media` | Pontos de montagem para mídia removível |
| `/var/log` | Logs. O primeiro suspeito no alerta de disco cheio |
| `/var/lib` | Estado persistente: base do `dpkg`, PostgreSQL, Docker |
| `/var/cache` | Recriável: `apt clean` libera espaço em 10 segundos |
| `/tmp` | Curta duração. Costuma ser **tmpfs = RAM**: some no boot |
| `/var/tmp` | Temporário que **sobrevive** à reinicialização |
| `/proc` | Processos e kernel. `/proc/self` é quem está lendo |
| `/sys` | Dispositivos e drivers: temperatura, `operstate` da placa |
| `/dev` `/run` | `null`, `zero`, `urandom`, discos · e o estado da execução atual |

em disco, estático · variável — cresce e enche · virtual — gerado pelo kernel, não ocupa disco

## A LÓGICA POR TRÁS · E ONDE INSTALAR O QUE

**ESTÁTICO · COMPARTILHÁVEL**
**`/usr`**
Pode ser montado somente-leitura ou servido por rede a várias estações.

**VARIÁVEL · COMPARTILHÁVEL**
**`/srv` · `/var/mail`**
Muda, mas faz sentido para outras máquinas.

**ESTÁTICO · LOCAL**
**`/etc` · `/boot`**
Específico desta máquina — por isso é o alvo prioritário de backup.

**VARIÁVEL · LOCAL**
**`/var/log` · `/run` · `/tmp`**
Volátil e local: exige rotação e monitoramento de espaço.

| VEIO DE ONDE | VAI PARA ONDE |
|---|---|
| Gerenciador de pacotes | `/usr/bin` — **não mexa** |
| Compilado do fonte | `/usr/local` |
| Terceiro autocontido | `/opt/<nome>` |
| Script só seu | `~/.local/bin` |
| Dados servidos | `/srv` ou `/var/www` |

## RESUMO RÁPIDO

- ✓ O caminho não diz em que disco o arquivo está — `findmnt --real` e `df -hT` dizem.
- ✓ Disco cheio: `sudo du -xh --max-depth=1 / | sort -rh | head`. O `-x` evita `/proc` e `/sys`.
- ✓ **Nunca escreva à mão em `/usr/bin`**: a próxima atualização sobrescreve sem avisar.
- ✓ `/proc`, `/sys` e `/dev` são "tudo é arquivo" na prática: leia com `cat`.

## NO KALI

Acabou o `/pentest/` do BackTrack: **ferramenta é pacote comum em `/usr/bin` — `dpkg -S /usr/bin/nmap` diz de qual pacote veio.** Os dados de cada uma ficam em `/usr/share/<ferramenta>` (scripts NSE, módulos do Metasploit) e as listas em `/usr/share/wordlists` — o `rockyou.txt.gz` vem compactado e precisa de `gunzip` uma vez. Ferramenta clonada de repositório vai para `/opt`. **Atenção ao `/tmp` em RAM:** salvar captura longa ali numa VM pequena derruba a máquina — use `~/lab/capturas`.

GUIA VISUAL 006

# ANATOMIA DE UM COMANDO

`rm arquivo com espaco.txt` *não apaga um arquivo: apaga três. Quem separou foi o shell, antes do programa existir.*

006
GUIAS LINUX

## A LINHA, PALAVRA POR PALAVRA

| `nmap` | `-sV` | `--top-ports` | `100` | `-oN` | `saida.txt` | `192.168.56.0/24` |
|---|---|---|---|---|---|---|
| COMANDO<br>o que será executado | OPÇÃO CURTA<br>um traço, uma letra | OPÇÃO LONGA<br>dois traços, palavra | VALOR<br>pertence à opção anterior | OPÇÃO<br>com valor | VALOR<br>arquivo de saída | OPERANDO<br>o alvo do trabalho |

**Para o shell, todas as palavras depois do comando são iguais.** Quem decide que `-sV` é opção e o resto é alvo é o **programa**, ao ler seu vetor de argumentos — por isso não existe sintaxe universal. Curtas sem valor se agrupam ( `ls -lah` ); `--` marca o **fim das opções** e salva de nome de arquivo que começa com traço: `rm -- -arquivo-estranho` . E há os rebeldes: `tar xzf` , `ps aux` , `dd if=` … `of=` … — sem traço nenhum.

## COMO O SHELL DECIDE O QUE EXECUTAR

1. **PALAVRA-CHAVE** — `if` , `for` , `while` — é sintaxe, não programa
2. **ALIAS** — Apelido seu ou da distro. **NÃO VALE EM SCRIPT**
3. **FUNÇÃO** — Definida na sessão ou no arquivo de init
4. **BUILTIN** — `cd` , `echo` , `export` — dentro do próprio shell
5. **PATH** — Arquivo em disco, na ordem dos diretórios

`type -a <comando>` **mostra qual das cinco venceu** — e explica por que o terminal e o script se comportam diferente. O **diretório atual não está no** `PATH` : por isso `./script.sh` . Não é inconveniência, é segurança — com `.` no `PATH` , um arquivo chamado `ls` num diretório gravável vira escalada de privilégio. **Nunca acrescente** `.` **ao** `PATH` **.**

## ZERO É SUCESSO — E É O QUE DÁ SENTIDO AO ENCADEAMENTO

| CÓDIGO DE SAÍDA | SIGNIFICA |
|---|---|
| **0** | Sucesso — o único valor bom |
| **1** | Erro genérico ( `grep` sem achar nada) |
| **2** | Erro de uso / arquivo inexistente |
| **126** | Existe, mas não é executável |
| **127** | Comando não encontrado — erro de digitação ou não instalado |
| **130** | Interrompido por `Ctrl+C` (128 + sinal 2) |

**`a && b`**
Roda **b** só se **a** deu certo. **O padrão para tudo que pressupõe o passo anterior.**

**`a || b`**
Roda **b** só se **a** falhou — o tratamento de erro.

**`a ; b`**
Roda **b** sempre, dê no que der.

**`a &`**
Roda em segundo plano e devolve o prompt na hora.

**`\rm · command rm`**
Ignora o alias e chama o programa real.

GUIA VISUAL 007

# OBTENDO AJUDA

*3h da manhã, link de 200 kbps, sem sinal de celular — e a resposta já está instalada na máquina.*

007
GUIAS LINUX

## A FONTE CERTA DEPENDE DA PERGUNTA

| A SUA DÚVIDA | ONDE PERGUNTAR | POR QUÊ |
|---|---|---|
| Sei o nome, quero a sintaxe agora | `comando --help` · `tldr comando` | Cabe na tela; `tldr` é só exemplo comentado |
| Sei o nome, quero a referência completa | `man comando` | Descreve **a versão instalada**, não a do autor do artigo |
| **Não sei o nome**, sei o que quero fazer | `apropos palavra` · `man -k palavra` | Busca na descrição de todas as páginas em disco |
| Quero o formato de um arquivo de configuração | `man 5 arquivo` | A seção 5 é dos formatos, não dos comandos |
| Por que este pacote se comporta assim? | `/usr/share/doc/<pacote>/` | `README.Debian` traz as decisões do empacotador |
| É builtin do shell (não tem arquivo) | `help cd` (Bash) · `run-help cd` (Zsh) | A doc vem do shell — confira antes com `type -a` |

## O MESMO NOME EXISTE EM SEÇÕES DIFERENTES

**1 COMANDOS DE USUÁRIO**
Tudo que se digita. `man 1 passwd` = trocar senha

**2 CHAMADAS DE SISTEMA**
A fronteira com o kernel — `open(2)`

**3 FUNÇÕES DE BIBLIOTECA**
C e glibc — `printf(3)`

**4 ARQUIVOS ESPECIAIS**
Dispositivos em `/dev`

**5 FORMATOS DE ARQUIVO**
A sintaxe da configuração. `man 5 passwd` = `/etc/passwd`

**6 JOGOS**
Herança histórica do Unix

**7 DIVERSOS E CONVENÇÕES**
Padrões e protocolos — `man 7 hier`

**8 ADMINISTRAÇÃO**
O que normalmente exige root — `man 8 mount`

**Sem o número, o `man` entrega a seção 1.** Por isso `man passwd` mostra o comando e `man 5 passwd` mostra o formato do arquivo. `whatis passwd` lista todas as seções em que o nome existe; `man -a passwd` percorre uma a uma. O número entre parênteses numa referência — `crontab(5)` — é exatamente isso: onde procurar.

## A GRAMÁTICA DO SYNOPSIS · E AS SEIS TECLAS DO `MAN`

```
ls [OPTION] ... [FILE] ...
cp [OPTION] ... SOURCE ... DIRECTORY
```

- `[colchetes]` opcional — pode omitir
- `reticências ...` pode repetir
- SEM `colchetes` obrigatório
- `{a|b}` escolha uma, obrigatória

Só com essa notação já se responde por que `ls` sozinho funciona e `cp arquivo` dá erro: o destino do `cp` não está entre colchetes.

| | |
|---|---|
| `espaço / b` | Avança / volta uma tela |
| `/texto` | **A tecla que salva:** busca para frente. Ninguém lê `man bash` inteiro |
| `n / N` | Próxima ocorrência / anterior |
| `g / G` | Início / fim da página |
| `h` | Ajuda do próprio visualizador |
| `q` | Sai |

GUIA VISUAL 008

# TERMINAL E SHELL

*Você configurou o prompt a tarde toda, abriu um SSH e tudo sumiu. Nada quebrou: o que você configurou não era o terminal.*

008
GUIAS LINUX

## DA TECLA AO PROGRAMA: QUATRO CAMADAS

| | | |
|---|---|---|
| **TECLADO E TELA** | Hardware, ou a janela do ambiente gráfico. | o que você aperta |
| **EMULADOR DE TERMINAL** | `qterminal`, `xterm`, `tmux`, PuTTY. Desenha glifos, traduz teclas, guarda a rolagem. **Não sabe o que é um comando.** | fonte · cores · abas · `TERM` |
| **PSEUDOTERMINAL DO KERNEL** | Aparece como `/dev/pts/N`. Faz o eco das teclas, junta a linha até o Enter e converte `Ctrl+C` em `SIGINT`. | é o kernel — por isso o `Ctrl+C` funciona com o programa travado |
| **SHELL** | `zsh`, `bash`, `dash`. Imprime o prompt, lê a linha, expande, resolve o nome e cria o processo. | prompt · histórico · alias · autocompletar |
| **PROGRAMA** | `ls`, `nmap`, `vim`. Nasce com três descritores já abertos apontando para o PTY. | stdin · stdout · stderr prontos |

**Cada problema de terminal pertence a exatamente uma camada.** No SSH o **emulador continua sendo o seu**, na sua máquina — o que mudou foi o shell, que agora roda no servidor e não tem a sua configuração. Lixo na tela ou `Error opening terminal` é a variável `TERM` errada (chave do banco **terminfo**): `export TERM=xterm` é a saída de emergência. E `ls --color=auto | cat` sai **sem cor** de propósito — o programa perguntou se a saída era um terminal e a resposta foi não.

## QUAL ARQUIVO O SHELL LÊ — E O QUE MUDA DO BASH PRO ZSH

**1. É SHELL DE LOGIN?**
SSH, terminal virtual de texto, `su -`, `sudo -i` → lê `/etc/profile` e `~/.zprofile`. **AQUI VAI VARIÁVEL DE AMBIENTE** (`PATH`, `EDITOR`): filho nenhum precisa relê-la.

**2. É INTERATIVO?**
Tem prompt e teclado — a aba nova do qterminal **É INTERATIVA E NÃO É LOGIN**. Lê `~/.zshrc` / `~/.bashrc`. **AQUI VAI ALIAS, FUNÇÃO E PROMPT.**

**3. NENHUM DOS DOIS?**
Um script. **NÃO LÊ NADA** — e é por isso que ele nunca enxerga os seus aliases. Errar entre 1 e 2 é a causa nº 1 de configuração que "não pega".

**APLICAR SEM FECHAR O TERMINAL**
`source ~/.zshrc` executa **NO SHELL ATUAL**. Executar o arquivo criaria um filho, aplicaria nele e o jogaria fora.

| SITUAÇÃO | BASH | ZSH |
|---|---|---|
| Config interativa | `~/.bashrc` | `~/.zshrc` |
| Curinga sem par | fica literal | erro `no matches` |
| Índice de array | `0` | `1` |
| Limpar cache | `hash -r` | `rehash` |
| Uso indicado | scripts | dia a dia |

## IDENTIFIQUE CADA CAMADA EM CINCO LINHAS

GUIA VISUAL 009

# NAVEGAÇÃO E CAMINHOS

*O comando do colega funciona na máquina dele e falha na sua. Não mudou o comando — mudou de onde ele foi executado.*

009
GUIAS LINUX

## MESMO DESTINO, DOIS CAMINHOS

```
/
├─ etc
│  └─ ssh
│     └─ sshd_config
├─ home
│  └─ kali
│     └─ projetos
│        └─ recon   ← pwd
└─ var
   └─ log
```

**ABSOLUTO — COMEÇA EM `/`**

`/etc/ssh/sshd_config`

Aponta para o mesmo arquivo **de qualquer lugar**. É o que se usa em script e em cron.

**RELATIVO — PARTE DO `PWD`**

`../../../etc/ssh/sshd_config`

Digita menos, mas **só vale a partir de `recon`**. Um `cd` antes e o mesmo texto aponta para outro lugar — ou para lugar nenhum.

**O diretório atual é um campo do *processo*, não do usuário nem do terminal.** Dá para vê-lo por fora em `/proc/<PID>/cwd` — útil para achar quem segura um sistema de arquivos que você quer desmontar. Daí saem três consequências: cada aba tem o seu; **o filho herda e nunca devolve** — um script que faz `cd` move a si mesmo e morre, o seu shell fica onde estava (por isso `source`); e é por isso que `cd` precisa ser *builtin*. Já `.` e `..` são **entradas reais** do sistema de arquivos, não sintaxe — é o que faz `./script.sh` rodar sem o diretório atual estar no `PATH`.

## A LINHA DO `LS -L`, CAMPO A CAMPO

| `-` | `rw-r--r--` | `1` | `root` | `root` | `2891` | `Jul 14 09:22` | `/etc/passwd` |
|---|---|---|---|---|---|---|---|
| TIPO<br>- arquivo · d dir · 1 link | PERMISSÕES<br>dono · grupo · outros | LINKS<br>quantos nomes apontam ao inode | DONO | GRUPO | TAMANHO<br>bytes (-h converte) | MODIFICAÇÃO<br>do conteúdo | NOME |

## OS ATALHOS E OS COMANDOS QUE VIRAM REFLEXO

| SÍMBOLO | SIGNIFICA |
|---|---|
| `.` | O diretório atual |
| `..` | O diretório pai |
| `~` | Seu diretório pessoal — **não expande dentro de aspas**; em script use `"$HOME"` |
| `~kali` | O diretório pessoal do usuário `kali` |
| `-` | O diretório anterior (guardado em `OLDPWD`), só com `cd` |
| `cd` | Sem argumento nenhum: vai para o seu `~` |

**`cd -`**
Alterna entre os dois últimos diretórios. O atalho que mais rende no dia a dia.

**`pushd dir · popd · dirs -v`**
Pilha de diretórios, para quando são mais de dois pontos na investigação.

**`ls -lah`**
O padrão de trabalho: longo, com ocultos, tamanho legível.

**`ls -ltr`**
Mais recente **por último**, coladinho no prompt. O reflexo de quem lê log.

**`ls -ld /var/log`**
A linha **do próprio diretório**, não o conteúdo — é o que responde "por que não consigo entrar aqui".

**`pwd -P · readlink -f · realpath`**
Resolvem links e `..` até o caminho físico. `pwd` sozinho mostra o caminho

GUIA VISUAL 010

# COPIAR, MOVER E APAGAR

`mv relatorio.md backups/` — *se* `backups` *não existir, você não moveu nada: renomeou o relatório.*

**010**
GUIAS LINUX

## O NOME E O CONTEÚDO SÃO COISAS SEPARADAS

### `cp a.txt b.txt`

`a.txt` → `inode 42`
`b.txt` → `inode 77`

Dois nomes, **dois conteúdos**. É a operação cara: lê e escreve tudo, ocupa o dobro, demora conforme o tamanho.

### `mv a.txt b.txt`

`b.txt` → `inode 42`
~~`a.txt`~~ `nome removido`

Só a **entrada de diretório** muda. Instantâneo e **atômico** mesmo com 40 GB — entre discos diferentes, porém, vira cópia + remoção.

### `rm a.txt`

~~`a.txt`~~ × `inode 42`
`links 1 → 0` → `blocos liberados`

Apaga um **nome**. O conteúdo só é liberado quando nenhum nome **e nenhum processo** apontam para ele — por isso apagar log em uso não devolve espaço.

## `MV ORIGEM DESTINO` — O DESTINO EXISTE?

### NÃO EXISTE
**→ renomeia para `destino`**

É o caso que morde: um diretório digitado errado vira um arquivo com nome de diretório, em silêncio.

### EXISTE E É DIRETÓRIO
**→ move para `destino/origem`**

O que você queria. **Termine o destino com barra** — `mv x backups/` — e o caso da esquerda vira erro em vez de surpresa.

### EXISTE E É ARQUIVO
**→ sobrescreve sem aviso**

Não há confirmação, não há lixeira. Antídotos: `-i` pergunta, `-n` nunca sobrescreve, `--backup=numbered` guarda o antigo.

## AS OPÇÕES QUE MUDAM O RESULTADO

| OPÇÃO | VALE PARA | O QUE FAZ |
|---|---|---|
| `-i` / `-I` | cp mv rm | Pergunta a cada arquivo / **uma vez** pelo lote |
| `-n` | cp mv | Nunca sobrescreve, e não pergunta |
| `-r` | cp rm | Recursivo — obrigatório para diretório |
| `-a` | cp | **Arquivamento**: `-r` + preserva data, dono, permissão e links |
| `-u` | cp mv | Só age se a origem for mais nova |
| `-p` | mkdir | Cria a cadeia toda e **não reclama se já existir** |

**`cmd > cfg.novo && mv cfg.novo cfg`**
Gere ao lado e mova por cima: ninguém jamais lê um arquivo pela metade.

**`rmdir vazio`**
Só remove diretório vazio — a recusa *é* o recurso de segurança.

**`rename -n 's/\.jpeg$/\.jpg/' *.jpeg`**
Renomeia em lote; o `-n` simula. Rode sempre com ele antes.

**`rsync -av --dry-run orig/ dest/`**
Árvore grande, retomável, só o que mudou. **Aqui a barra final muda tudo**: com ela copia o conteúdo, sem ela cria `dest/orig`.

**`stat arq` · `touch -d "2020-01-01" arq`**
**mtime** conteúdo · **atime** leitura · **ctime** metadados. O `ctime` não se falsifica pelo `touch`: `ctime` mais novo que `mtime` denuncia carimbo alterado.

**Não existe lixeira, e o `rm -rf` não pede licença.** Ritual antes de todo comando destrutivo: **1.** `pwd` — onde estou? **2.** `ls -la alvo/` — é isto mesmo? **3.** só então o `rm`. Ou troque o passo 2 por `echo rm -rf "$DIR"/*`: o `echo` imprime a linha **já expandida**, sem executar. Quatro linhas que já destruíram sistemas reais: `rm -rf / caminho/` (o espaço apaga a raiz) · `rm -rf "$DIR"/*` com `$DIR` vazia vira `rm -rf /*` · `cd /alvo ; rm -rf *` se o `cd` falhar (use `&&`) · `rm -rf .*`. O `--preserve-root` é padrão, mas cobre só a raiz literal — `/*` passa.

GUIA VISUAL 011

# LER O CONTEÚDO

*`cat` num log gigante arrasa a rolagem; `cat` num binário embaralha o terminal. Nenhum dos dois é acidente — o `cat` não serve para ver arquivo.*

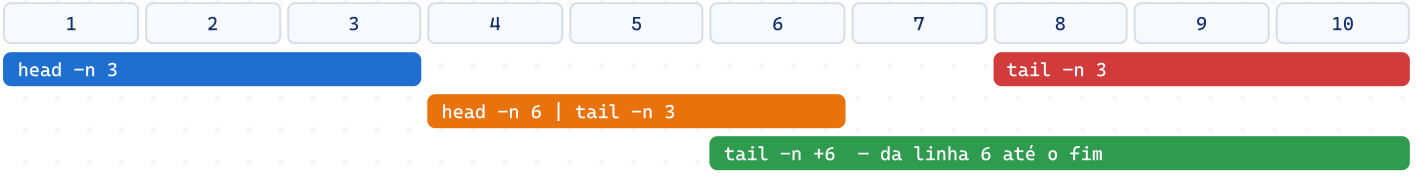
011
GUIAS LINUX

## RODE `FILE` ANTES DE ABRIR — E ESCOLHA A FERRAMENTA

| TEXTO CURTO | TEXTO GRANDE | SÓ AS PONTAS | CRESCENDO | COMPRIMIDO | BINÁRIO |
|---|---|---|---|---|---|
| **`cat`** | **`less`** | **`head`** **`tail`** | **`tail -F`** **`less +F`** | **`zcat zless zgrep`** | **`strings`** **`xxd`** |
| Cabe numa tela. Só aí. | Carrega sob demanda: 10 GB abrem na hora. | Primeiras ou últimas linhas. | Acompanha o arquivo em tempo real. | Lê o `.gz` sem descomprimir. | **Nunca `cat`** — conserto: `reset`. |

**O `cat` *concatena* — exibir é só o que sobra quando você dá um arquivo e não redireciona.** O `file` não confia na extensão: lê os primeiros bytes e compara com um banco de assinaturas. Num binário, os bytes que por acaso formam sequências de escape VT100 são **obedecidos** — o terminal não travou, ele mudou de fonte e de mapa de caracteres; digite `reset` às cegas e Enter. E o `cat -A` é diagnóstico de verdade: revela `^M` (fim de linha do Windows — a causa do `bad interpreter: /bin/bash^M`), `^I` (tabulação onde você achava que havia espaços) e `$` (espaço sobrando no fim da linha).

## O SINAL DE MAIS MUDA TUDO: AS FAIXAS DE `HEAD` E `TAIL`




## DEZ TECLAS DO `LESS` E OS REFLEXOS QUE SOBRAM

| TECLA | FAZ |
|---|---|
| `Espaço` / `b` | Avança / volta uma tela |
| `g` / `G` | Início / fim do arquivo |
| `/texto` · `n` `N` | Busca; próxima e anterior ocorrência |
| `&texto` | **Mostra só as linhas que casam** — um grep interativo sem sair do arquivo (`&` + Enter volta ao todo) |
| `F` | Acompanha o crescimento; `Ctrl+C` sai do modo **sem perder o histórico** |
| `m` +letra · `'` +letra | Marca a posição e volta a ela |
| `v` · `h` · `q` | Abre no `$EDITOR` · ajuda · sai |

**`tail -f` × `tail -F`**
O `-f` segue o **inode**: depois que o `logrotate` renomeia o arquivo, ele fica olhando o antigo para sempre. O `-F` segue o **nome** e reabre. Em servidor é sempre `-F`.

**`tail -F auth.log | grep --line-buffered "Failed"`**
Sem `--line-buffered` o grep segura kilobytes e você fica minutos achando que travou.

**`less -R` · `export LESS='-R -i -M'`**
`-R` interpreta as cores do `nmap`; sem ele você lê `ESC[1;32m` em cada linha. É o mesmo paginador do `man`.

**`grep erro arquivo`**
Não `cat arquivo | grep`: todo comando que aceita nome de arquivo dispensa o `cat` — é um processo e uma cópia de dados a menos.

**`tac` · `batcat` · `jq . | less -R`**
`tac` lê do fim para o começo. `batcat` (o `bat` do Debian) colore por sintaxe — ótimo para código, dispensável para dados que seguem num cano.

### RESUMO RÁPIDO

- `file` antes de abrir — a extensão mente, os bytes não.
- Texto grande → `less`; só as pontas → `head` / `tail`.
- `tail -F` para log que gira; `-f` segue o inode.
- Binário → `strings` ou `xxd`; nunca `cat`.
- `cat -A` revela `^M` (CRLF) e `$` (espaço sobrando).
- `reset` às cegas e Enter quando o terminal quebrar.

### NO KALI

`batcat` (o `bat` do Debian) já vem instalado. `jq` também. Para `xxd` e `strings`, instale `xxd` e `binutils`.

GUIA VISUAL 012

# REDIRECIONAMENTO E PIPES

*Você guardou a varredura com `>` e os erros sumiram com o buffer da tela. Trocar para `2>&1 > arq` **não** conserta.*

012
GUIAS LINUX

## TODO PROCESSO NASCE COM TRÊS CANAIS ABERTOS

**`comando`**

| | | |
|---|---|---|
| 0 · stdin | ← | teclado |
| 1 · stdout | → | tela |
| 2 · stderr | → | tela |

Resultado e erro **misturados** na mesma tela — e é só a numeração que os separa.

**`comando > saida.txt`**

| | | |
|---|---|---|
| 0 · stdin | ← | teclado |
| 1 · stdout | → | saida.txt |
| 2 · stderr | → | tela |

**Só o descritor 1 mudou de destino.** Redirecionar é reapontar um canal *antes* de o programa começar — e o programa nunca percebe.

## A MESMA DUPLA, DUAS ORDENS, RESULTADOS OPOSTOS

**`comando > log.txt 2>&1`**

1. **`> log.txt`** — o descritor **1** passa a apontar para o arquivo.
2. **`2>&1`** — o **2** copia o destino *atual* do 1: o arquivo.

✓ resultado **e** erro no arquivo

**`comando 2>&1 > log.txt`**

1. **`2>&1`** — o **2** copia o destino atual do 1: a **tela**.
2. **`> log.txt`** — só o **1** muda para o arquivo.

✗ resultado no arquivo, erro **na tela**

`2>&1` **copia o destino *daquele instante* — não é vínculo.** O shell lê os redirecionamentos da esquerda para a direita, então a regra é: **o `2>&1` vem por último**, ou use `&>`. A ordem "errada" tem um uso legítimo e elegante: `comando 2>&1 > /dev/null | grep falha` manda **só os erros** para o cano, porque o `stderr` foi copiado para o cano antes de o `stdout` ser descartado.

## AS FORMAS QUE SE USAM TODO DIA

| FORMA | O QUE FAZ |
|---|---|
| `>` / `>>` | stdout sobrescrevendo / acrescentando |
| `2>` / `2>>` | Só o stderr |
| `&>` / `&>>` | Os dois no mesmo arquivo (Bash e Zsh; em `sh` use `> a 2>&1`) |
| `<` | Lê do arquivo. `wc -l < f` imprime só o número, sem o nome |
| `<< FIM` | *Here-doc*: texto até o marcador. Entre aspas (`<< 'FIM'`) não expande variável |
| `<<<` | *Here-string*: uma linha, sem `echo` e sem cano |
| `\|` / `\|&` | Cano só do stdout / com o stderr junto |

**`a | b | c`**
Os três rodam **ao mesmo tempo**, não em sequência: o kernel põe um buffer entre cada par. É o que permite processar 100 GB com poucos MB de RAM.

**`set -o pipefail` · `${PIPESTATUS[@]}`**
Por padrão o status do pipeline é o do **último** comando: `falso | tee f` devolve **0**. Esse detalhe já mascarou backup quebrado por meses.

**`echo linha | sudo tee -a /etc/hosts > /dev/null`**
A resposta certa para `sudo ... > arquivo`, que falha porque **quem redireciona é o seu shell**, não o sudo. O `tee` grava e repassa.

**`diff <(ls /etc) <(ls /etc/apt)`**
Substituição de processo: entrega um caminho como `/dev/fd/63` onde o comando exige **nome de arquivo**. Bash e Zsh, não `sh`.

**`grep --line-buffered` · `stdbuf -oL` · `unbuffer`**
Filtro ao vivo que "trava" é *buffering*: a libc usa buffer por linha na tela e **por bloco** no cano. Não é bug — é otimização atrapalhando.

GUIA VISUAL 013

# CURINGAS E EXPANSÕES

*O programa **nunca viu o asterisco**. Quem procurou os arquivos, montou a lista e escolheu a ordem foi o shell — antes de o comando existir.*

013
GUIAS LINUX

## OITO ETAPAS, SEMPRE NESTA ORDEM, NA MESMA LINHA

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|
| CHAVES `{a,b}` | TIL `~` | VARIÁVEIS `$VAR` | COMANDOS `$(cmd)` | ARITMÉTICA `$((1+2))` | DIVISÃO EM PALAVRAS IFS | GLOBBING `* ? [ ]` | REMOÇÃO DE ASPAS `" '` |

**A etapa 1 gera texto do nada** — não consulta o disco, e por isso `mkdir {a,b,c}` funciona com nada existindo. **A etapa 7 olha o disco** e só produz o que existe de fato. E como o globbing vem **depois** das variáveis, um padrão guardado numa variável é expandido ao ser usado *sem aspas*: `PADRAO="*.txt"` → `echo $PADRAO` lista os arquivos, `echo "$PADRAO"` imprime o texto. O programa só roda **depois das oito etapas**, com a lista final de palavras — e a conferência custa dois segundos: `echo mv *.txt arquivo/` .

## QUEM CASA COM QUÊ

| ARQUIVO NO DIRETÓRIO | *.TXT | LOG?.TXT | LOG[12].TXT | * | .* |
|---|---|---|---|---|---|
| `log1.txt` | ✓ | ✓ | ✓ | ✓ | × |
| `log2.txt` | ✓ | ✓ | ✓ | ✓ | × |
| `log10.txt` | ✓ | × `?` é **um** caractere | × | ✓ | × |
| `nota.md` | × | × | × | ✓ | × |
| `.oculto` | × | × | × | × **o `*` não pega oculto** | ✓ |

## CHAVES GERAM · CURINGAS ENCONTRAM · E ONDE ISSO MACHUCA

**`cp app.conf{,.bak}`**
Vira `cp app.conf app.conf.bak` . O elemento vazio produz o nome original e o com sufixo — **o backup mais curto que existe**.

**`mkdir -p proj/{doc,src}/{in,out}`**
Produto cartesiano: quatro diretórios numa linha. Chaves não consultam o disco, então criam o que ainda não existe.

**`touch teste-{01..10}.txt` · `echo {0..20..5}`**
Faixas com zero à esquerda preservado e com passo. `{10..1}` conta ao contrário.

**`diff arquivo.{antigo,novo}`**
Digite o nome uma vez só. E `ls *.{txt,md}` vira `ls *.txt *.md` — cada padrão passa pelo globbing depois.

**`ls *(.om[1,3])` · `ls **/*(.Lm+50)`**
Qualificadores de glob do **Zsh**: os 3 mais recentes, os maiores que 50 MB. É `find` embutido na sintaxe — e não existe no Bash.

**Sem correspondência: Bash ≠ Zsh**
Bash entrega o **texto literal** ao comando; Zsh aborta com `no matches found` . Em laço Bash, `shopt -s nullglob` é obrigatório — sem ele, `for f in *.log` roda **uma vez** com `$f` valendo `*.log` .

**`cp "$f" /tmp/` — nunca `cp $f /tmp/`**
O glob entrega o nome certo; quem quebra `meu arquivo.txt` em dois é a **etapa 6**, na variável sem aspas.

**`rm -- *` · `rm ./*`**
Um arquivo chamado `-rf` no diretório transforma `rm *` em `rm -rf resto` . O `--` marca o fim das opções.

**`*` não cruza a barra e ordena como texto**
`**` desce a árvore ( `shopt -s globstar` no Bash — sem ele vira um `*` comum, **em silêncio**). E a ordem é `log1 log10 log2` : quem ordena é o shell, alfabeticamente.

**`find . -name '*.log' -print0 | xargs -0 rm`**
Para `Argument list too long` e para nomes com espaço, aspas ou

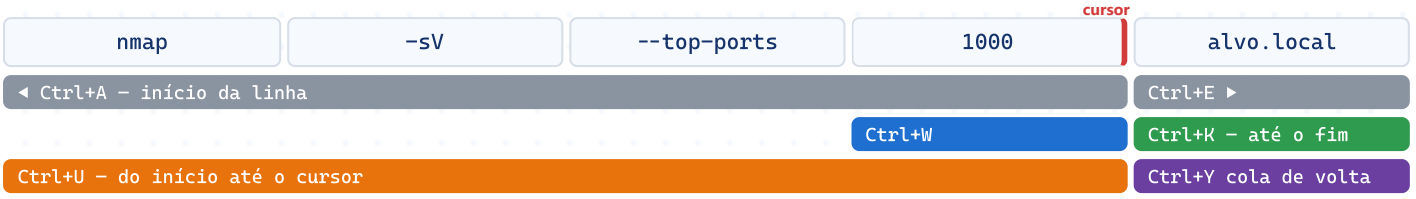
GUIA VISUAL 014

# HISTÓRICO E ATALHOS

*Errou o fim de uma linha de noventa caracteres? Não segure a seta nem apague tudo. E, no fim das contas, nem precisava redigitar: `!!` repete.*

014
GUIAS LINUX

## A LINHA É UM EDITOR — E O QUE CADA ATALHO APAGA DEPENDE DO CURSOR




## POR QUE O HISTÓRICO ÀS VEZES SOME

**1 — VOCÊ DIGITA**
A linha entra na sessão assim que você aperta Enter.

→

**2 — LISTA EM MEMÓRIA**
Limite: `HISTSIZE`. É daqui que vêm `Ctrl+R` e a seta para cima.

→

**3 — ARQUIVO, SÓ AO SAIR**
`HISTFILE`, limite `SAVEHIST`. **Com duas abas abertas, a última a fechar sobrescreve** — e apaga o histórico da outra.

→

**4 — A CORREÇÃO**
`setopt INC_APPEND_HISTORY SHARE_HISTORY EXTENDED_HISTORY` (Zsh) · `shopt -s histappend` + `PROMPT_COMMAND='history -a'` (Bash).

## O QUE SE APRENDE UMA VEZ E RENDE PELO RESTO DA VIDA

| ATALHO | FAZ |
|---|---|
| `Alt+B` / `Alt+F` | Anda uma **palavra** para trás / para frente |
| `Alt+.` | **Insere o último argumento** do comando anterior — repita para percorrer os anteriores |
| `Alt+D` | Apaga a palavra *depois* do cursor |
| `Ctrl+T` / `Alt+T` | Troca os dois caracteres / as duas palavras em torno do cursor |
| `Ctrl+_` | Desfaz a última edição |
| `Ctrl+L` | Limpa a tela mantendo a linha em edição |
| `Ctrl+X Ctrl+E` | Abre a linha no `$EDITOR`; ao salvar, o shell executa |
| `Ctrl+R` | **Busca reversa**. De novo = ocorrência anterior · seta = editar · `Ctrl+G` = cancelar |

**`apt update → sudo !!`**
`!!` repete a linha inteira. O idioma mais usado do histórico.

**`mkdir -p a/b/c → cd !$`**
`!$` é o último argumento · `!^` o primeiro · `!*` todos · `!nmap` o último que começou assim.

**`^sysog^syslog`**
Repete o anterior trocando a **primeira** ocorrência do texto. Conserta erro de digitação sem redigitar nada.

**`!!:p`**
**Mostra sem executar**, e deixa a linha na seta para cima. Use antes de todo `!!` que envolva `sudo` ou `rm`.

**`Ctrl+S` congela · `Ctrl+Q` descongela**
O motivo nº 1 de "meu terminal travou": a mão vai no `Ctrl+S` de salvar e o controle de fluxo para a **exibição** — o programa segue rodando. `stty -ixon` no `~/.zshrc` desliga o susto de vez. (`Ctrl+C`, `Ctrl+Z`, `Ctrl+D` e estes dois são do **driver do terminal**, não do shell — por isso funcionam com o programa ocupado.)

**`alias ll='ls -lah'` · `mkcd() { mkdir -p "$1" && cd "$1"; }`**
Alias para o que se repete; **função** quando precisa de argumento — alias não recebe. E `fzf` troca o `Ctrl+R` por busca difusa no histórico inteiro.

GUIA VISUAL 015

# TIPOS, INODES E A ÁRVORE

*O disco tem 40% livre e o sistema insiste:* `No space left on device`.

015
GUIAS LINUX

## O NOME NÃO FAZ PARTE DO ARQUIVO

### DIRETÓRIO /HOME/KALI

| NOME | INODE |
|---|---|
| relatorio.txt | 917 |
| notas.md | 4432 |
| backup.txt | 917 |

Só nome e número.
Nenhum dado do arquivo aqui.

→

### INODE 917 — OS METADADOS

**inode 917**

tipo: **comum** — permissões: **rw-r--r--**
dono: **kali:kali** — tamanho: **2048**
atime · mtime — ctime · btime
**links: 2 → dois nomes, um arquivo**

Tudo sobre o arquivo — menos o nome.

→

### BLOCOS DE DADOS

`0x1A` `0x1B`
`0x2F` `0x30`

O conteúdo em si,
espalhado pelo disco.

## OS SETE TIPOS — O 1º CARACTERE DO `LS -L`

| `-` | `d` | `l` | `c` | `b` | `p` | `s` |
|---|---|---|---|---|---|---|
| COMUM | DIRETÓRIO | LINK SIMB. | CARACTERE | BLOCO | PIPE FIFO | SOCKET |
| /etc/passwd | ls -ld /etc | /usr/bin/vi | /dev/null | /dev/sda | mkfifo cano | /run/*.sock |

## NA PRÁTICA

kali@kali: ~

```
$ cp relatorio.txt copia.txt ; ls -li *.txt
 917 -rw-r--r-- 2 kali kali 2048 relatorio.txt
4501 -rw-r--r-- 1 kali kali 2048 copia.txt
# cp cria inode novo; mv mantem o mesmo
$ df -h /   Use% 40%   # bytes: sobra
$ df -i /   IUse% 100%  # inodes: acabaram
$ file -b foto.jpg
ELF 64-bit LSB executable # nao e foto
```

| MARCA | MUDA QUANDO |
|---|---|
| `atime` | o conteúdo é **lido** |
| `mtime` | o **conteúdo** muda |
| `ctime` | o **inode** muda: chmod, dono, nome |
| `btime` | criação — nunca muda |

**Inodes não se acrescentam depois.** A quantidade de um ext4 é fixada em `mkfs.ext4 -N` na formatação — reformatar apaga a partição inteira. Só faça com backup pronto e em disco de laboratório (arquivo-imagem via `losetup` ou VM descartável). Btrfs e XFS alocam inodes dinamicamente e não sofrem disso.

### RESUMO RÁPIDO

- ✓ O nome mora no diretório; tipo, dono, tempos e ponteiros moram no **inode**.
- ✓ Renomear é instantâneo; `cp` cria inode novo, `mv` reaproveita.
- ✓ Apagar arquivo ainda aberto não devolve espaço — o inode só morre com links = 0.
- ✓ `ctime` é *change*, não *creation*: o `touch` não o falsifica.
- ✓ Diretório vazio já tem 2 links (o nome no pai e o `.`); +1 por

### NO KALI

A raiz é **ext4**, de inodes fixos: varredura que grava um arquivo por alvo ( `gobuster` , `nuclei` ) esgota inodes muito antes do espaço — se `df -h` não bate com o erro, rode `df -i` . Em `/usr/share/wordlists` quase tudo é link: use `file -L` . E `stat` é forense — `ctime` de agora com `mtime` de 2015 denuncia arquivo adulterado. Ferramenta que reclama de `/dev/net/tun` ou `/dev/bus/usb` quase nunca está quebrada: é grupo do usuário.

GUIA VISUAL 016

# LINKS SIMBÓLICOS E HARD LINKS

*O `cat` diz No such file or directory — mas o arquivo está ali, dois diretórios adiante.*

016
GUIAS LINUX

## UM APONTA PARA O INODE, O OUTRO PARA UM CAMINHO

### HARD LINK

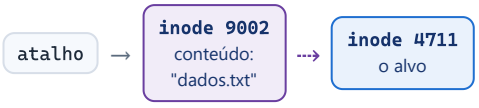
`ln dados.txt copia.txt`




- ✓ Dois nomes legítimos: **não existe "o original"**.
- ✓ Sobrevive a renomear/mover o outro nome.
- ✓ Some só quando o contador zera.
- ✗ Não atravessa sistema de arquivos.
- ✗ Não funciona para diretório.

### LINK SIMBÓLICO

`ln -s dados.txt atalho`




- ✓ Arquivo próprio, tipo `l` ; tamanho = nº de caracteres do caminho.
- ✓ Atravessa sistemas de arquivos e aceita diretório.
- ✓ Pode apontar para o que ainda não existe.
- ✗ Alvo saiu do lugar → link pendurado.
- ✗ Não conta no contador de links do alvo.

## AS TRÊS ARMADILHAS DO `LN`

### ORDEM DOS ARGUMENTOS

`ln -s ALVO LINK` — origem primeiro, como no **cp**. Se **LINK** for um diretório existente, o link nasce *dentro* dele com o nome-base do alvo.

### -SFN, NUNCA SÓ -SF

Se o link já aponta para um diretório, `ln -sf` **entra** nele e cria o link lá dentro. O `-n` trata o link como arquivo e substitui de fato.

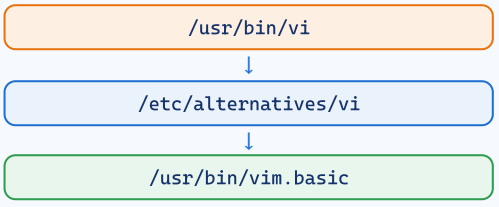
### RELATIVO × ABSOLUTO

O relativo resolve a partir de **onde o link está**, não de onde você o criou. `ln -sr` calcula o `../` sozinho — ideal em árvore que será movida.

## NA PRÁTICA

```
kali@kali: ~
$ ln dados.txt copia.txt # hard link
$ stat -c '%n i=%i links=%h' dados.txt copia.txt
dados.txt i=4711 links=2
copia.txt i=4711 links=2
$ ln -s dados.txt atalho ; ls -l atalho
lrwxrwxrwx 1 kali kali 9 atalho -> dados.txt
$ find . -xtype l -printf '%p -> %l\n'
./config.yaml -> ../antigo/config.yaml
$ namei -l /usr/bin/vi # cada etapa e a permissao
```

### A CADEIA DO `VI` NO KALI




Só o elo do meio muda em `update-alternatives --config editor` . Nada em `/usr/bin` é tocado — e a atualização do pacote não desfaz a escolha.

`chown -RL` **pode arruinar o sistema.** O `-L` segue os links e leva a mudança de dono para fora da árvore — um link para `/etc` dentro de `/var/www` basta. Use `chown -Rh` (muda o dono do link, não do alvo) e, antes de qualquer `-R` em diretório que você não criou, liste o que há lá dentro: `find /var/www -type l -ls` .

### RESUMO RÁPIDO

- ✓ Hard link = outro nome para o mesmo inode; simbólico = arquivo com um caminho dentro.
- ✓ `-R` não segue, `-L` segue, `-H` só o da linha de comando.
- ✓ `ln -s ALVO LINK` — origem primeiro, como no **cp**.
- ✓ `ln -sf` **entra** no diretório; `-n` substitui o link.
- ✓ `-sr` calcula o caminho relativo sozinho.
- ✓ `chown -RL` pode arruinar o sistema.

### NO KALI

`/usr/share/wordlists` é quase todo feito de links para os diretórios dos pacotes — `namei -l` mostra onde de fato vão parar, e um `apt autoremove` agressivo deixa links pendurados

GUIA VISUAL 020

# PERMISSÕES DE ARQUIVO

*Quem pode ler, escrever e executar — e por que `chmod 777` não resolve.*

020
GUIAS LINUX

## ANATOMIA DA SAÍDA DO `LS -L`




| TIPO | DONO | GRUPO | OUTROS |
|---|---|---|---|
| arquivo | 6 | 4 | 4 |

`r` = ler (4) `w` = escrever (2) `x` = executar (1) → `-rw-r--r--` = `644`

## A REGRA QUE QUASE NINGUÉM APRENDE

### SÓ UMA CLASSE SE APLICA

O Linux checa na ordem: é o **dono**? senão, está no **grupo**? senão, **outros**. Para na primeira que casa.

### EM ARQUIVO

**r** lê o conteúdo · **w** altera o conteúdo · **x** executa como programa.

### EM DIRETÓRIO

**r** lista nomes · **w** cria/remove itens · **x** *atravessa* o diretório. Sem **x**, nada dentro é acessível.

## AS DUAS NOTAÇÕES DO `CHMOD`

### SIMBÓLICA

- ✓ Alvo: `u` dono · `g` grupo · `o` outros · `a` todos
- ✓ Operador: `+` adiciona · `-` remove · `=` define
- ✓ Ex.: `chmod u+x script.sh`

### OCTAL

- ✓ Soma dos bits por classe: **r=4, w=2, x=1**
- ✓ Três dígitos: dono · grupo · outros
- ✓ Ex.: `chmod 755 script.sh`

## NA PRÁTICA

| OCTAL | SIMBÓLICO | USO TÍPICO |
|---|---|---|
| `644` | `-rw-r--r--` | Arquivo comum (todos leem, só o dono edita) |
| `600` | `-rw-------` | Arquivo privado — chave SSH, segredo |
| `755` | `-rwxr-xr-x` | Script/binário e diretórios navegáveis |
| `700` | `-rwx------` | Diretório pessoal, script só seu ( `~/.ssh` ) |

**chmod 777 não é solução de nada.** Abre leitura, escrita e execução para *qualquer* usuário da máquina. Se o acesso falha, o culpado quase sempre é a falta de `x` em um diretório do caminho — não a permissão do arquivo.